

ScriptLens — Application Blueprint

Product	ScriptLens (customer v3 — structure-only)
Document type	Full application blueprint
Date	July 2026
Status	Local / pre-deploy; customer v1 features implemented
Audience	Founders, engineers, investors, contractors
Canonical specs	docs/ARCHITECTURE_v3_STRUCTURE.md, docs/SCRIPTLENS_STATUS_REPORT.md

1. One-sentence product

Upload your screenplay, see loose (orphan) scenes, and preview what breaks if you cut or rewrite — without changing your original file.

2. What ScriptLens is (and is not)

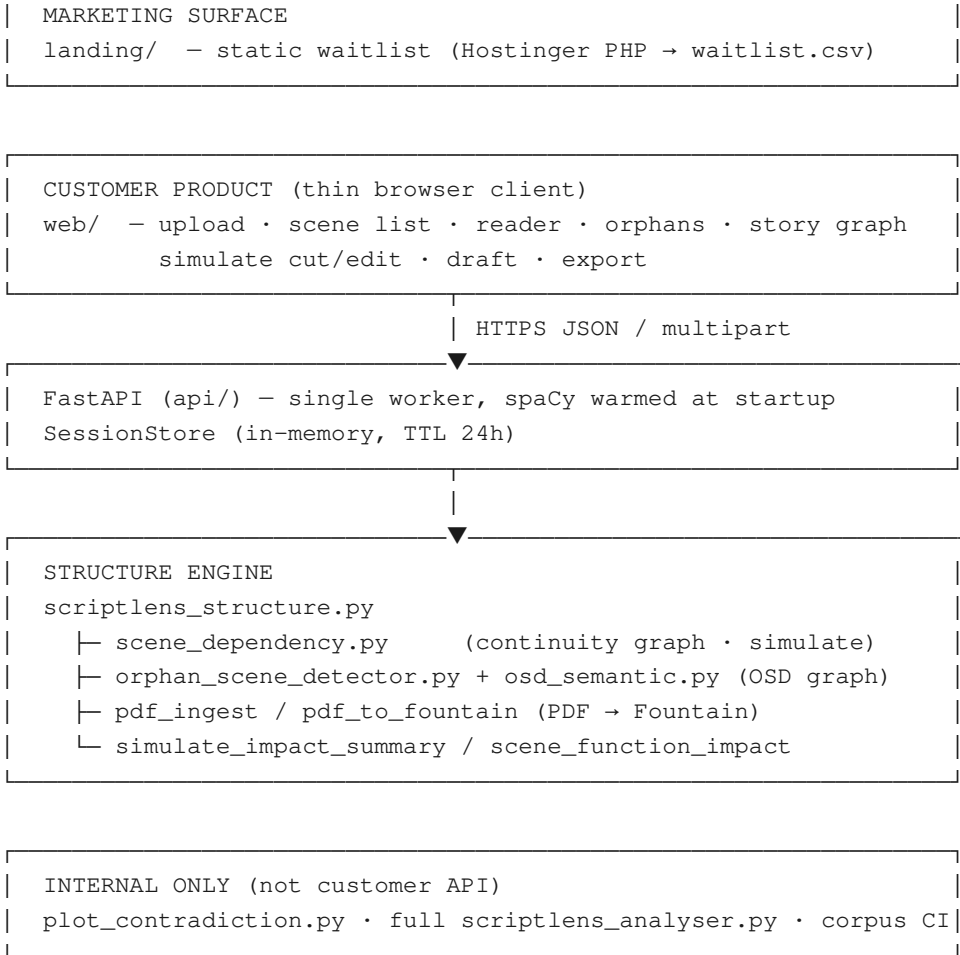
Is

Structural editing intelligence	Scene graph + impact previews
Orphan detection	Scenes with weak ties in the story graph (OSD)
Simulate cut	Preview downstream breakages if a scene is removed
Simulate edit	Preview dependency edge changes if scene text is rewritten
Draft workflow	Delete / apply edit / undo / export Fountain (working copy only)
PDF ingest	Text PDF → auto-cleaned Fountain (<i>refined</i>)

Is not (customer v1)

- Not a screenplay editor (writers stay in Final Draft / WriterDuet / etc.)
- Not a continuity / plot-contradiction product in the UI (engine kept for internal CI only)
- Not production breakdown (cast/props scheduling)
- Not a Chrome extension (planned v3.1+)
- Not auth / billing / multi-tenant cloud yet

3. System overview



Design principles

Thin client	Browser uploads and displays; server does all NLP/graph work
Non-destructive	Original upload never modified; draft is a separate working copy
Deterministic core	Explainable graph edges — no LLM in customer v1 path
Solo-deployable	Hostinger VPS + static UI + one FastAPI process
Scope lock	Contradictions out of customer UI until a later product decision

4. User journeys

4.1 Waitlist (pre-product)

- 1. Visitor opens marketing landing page.
- 2. Submits email → stored in waitlist.csv (Hostinger) or local serve.py.
- 3. Thank-you confirmation.

4.2 Structure workspace (customer v1)

- 1. Open web app → upload Fountain or PDF.
- 2. Server parses, builds graphs, returns script_id + scene list + orphan count.

3. Writer browses scenes, opens **story graph**, reviews **orphans**.
4. Selects a scene → **Simulate cut** or **Simulate edit** (preview only).
5. Optionally **Delete** / **Apply edit** on draft → **Undo** / **Export** Fountain.
6. Session expires after 24 hours (in-memory).

5. Technology stack

Web workspace	Static HTML / CSS / vanilla JS (<code>web/</code>)
Marketing waitlist	Static HTML / CSS + PHP (<code>landing/</code>)
API	FastAPI + Uvicorn (<code>api/</code> , <code>run_api.py</code>)
NLP	spaCy <code>en_core_web_sm</code> (shared singleton)
Graphs	NetworkX
PDF	PyMuPDF / project PDF pipeline
Sessions	In-memory dict + TTL (no Redis in v3)
Tests	pytest (~236+), GitHub Actions CI
Deploy target	Hostinger VPS (API) + Hostinger static/PHP (landing)

6. Repository map (product-facing)

<code>api/main.py</code>	FastAPI app, CORS, spaCy warmup, mount <code>web/</code>
<code>api/routes/upload.py</code>	Multipart upload + analyse
<code>api/routes/scripts.py</code>	Metadata, orphans, orphan-graph, scene body
<code>api/routes/simulate.py</code>	Simulate cut / edit
<code>api/routes/draft.py</code>	Delete, apply-edit, undo, export
<code>api/routes/health.py</code>	Liveness
<code>api/sessions.py</code>	Session store + TTL
<code>api/schemas.py</code>	Pydantic contracts
<code>scriptlens_structure.py</code>	Structure-only orchestration (no contradictions)
<code>scene_dependency.py</code>	Parse, continuity graph, simulate cut/edit
<code>orphan_scene_detector.py</code>	OSD orphan classification

osd_semantic.py	Semantic edges for OSD
pdf_ingest.py / pdf_to_fountain.py	PDF → cleaned Fountain
web/	Customer workspace UI
landing/	Waitlist marketing page
run_api.py	Local API entrypoint
run_scriptlens.py	CLI (use <code>--structure-only</code> for product path)
plot_contradiction.py	Internal CI only

7. API surface (implemented)

Base: `/api`

GET	<code>/health</code>	Liveness
POST	<code>/upload</code>	Upload + structure analysis
GET	<code>/scripts/{id}</code>	Script metadata
GET	<code>/scripts/{id}/orphans</code>	Orphan list + types/reasons
GET	<code>/scripts/{id}/orphan-graph</code>	Story graph payload
GET	<code>/scripts/{id}/scenes/{scene_id}</code>	Scene body for reader
POST	<code>/scripts/{id}/simulate/cut</code>	Simulate remove scene
POST	<code>/scripts/{id}/simulate/edit</code>	Simulate rewrite scene
POST	<code>/scripts/{id}/draft/delete</code>	Delete scene from draft
POST	<code>/scripts/{id}/draft/apply-edit</code>	Apply rewrite to draft
POST	<code>/scripts/{id}/draft/undo</code>	Undo last draft change
GET	<code>/scripts/{id}/draft/export</code>	Download draft <code>.fountain</code>

Static UI served from `/` when `web/` is present.

Upload response (summary)

- `script_id`, `filename`, `scene_count`, `orphan_count`
- `structure_mode`: `full` | `limited`
- `scenes[]` (`id`, `number`, `heading`)
- `draft_revision`, ingest warnings / PDF conversion metadata

8. Core engines (two graphs by design)

8.1 Continuity / dependency graph (`scene_dependency.py`)

- **Nodes:** scenes
- **Edges:** earlier → later narrative dependencies (characters, objects, locations, facts)
- **Used for:** simulate cut, simulate edit, draft impact

Simulate cut: find descendants of removed scene; rank by path weight; return impacted scenes + explanations.

Simulate edit: splice modified text → rebuild graph → diff edges before/after.

8.2 Orphan Scene Detector graph (OSD)

- Signals: Character (C), Location/spatial (L), Prop (P), Semantic (E)
- Link threshold typically **0.20**
- Orphan types: e.g. `hard`, `subplot_chain` (+ exemptions for prologue/montage/flashback patterns in fixtures)
- **Used for:** orphan badges, story graph viewer

8.3 Internal: plot contradiction

- Module: `plot_contradiction.py`
- **Not** called by v3 upload API or web UI
- Used for planted corpus CI (high recall gate)

9. Data & session model

```
AnalysisSession
  script_id          # derived from content hash prefix
  screenplay_text     # canonical Fountain
  scenes[]           # parsed SceneBlock list
  engine             # SceneDependencyEngine (+ OSD artifacts)
  structure_mode      # full | limited
  draft_state        # revision stack for undo/export
  created_at         # TTL clock (default 24h)
```

Privacy (v3): scripts live in server RAM only; no long-term DB; TTL expiry deletes session.

10. Input formats

Fountain / <code>.txt</code> / <code>.md</code>	Full — preferred
Text-based PDF	Full — auto <code>refined</code> cleanup
<code>.docx</code>	Converter exists; upload API not wired yet
Final Draft <code>.fdx</code>	Not native — export to Fountain/PDF first
Scanned / image PDF	Friendly failure (no OCR in v1)

11. Web workspace (UI blueprint)

Upload screen	Drop Fountain/PDF; privacy note
Left: scene list	Numbers, headings, orphan badges
Center: reader	Scene body; simulate ghost states
Right: panels	Orphans, simulate impact, draft actions
Story graph	OSD nodes/edges visualization
Banner	Full vs limited structure mode

Constraint: no client-side spaCy/PDF parsing — suitable for 4 GB laptops.

12. Marketing waitlist (landing/)

index.html	Black/gold Oscars-inspired waitlist page
styles.css	Theme + scene slugline watermarks
submit.php	Hostinger: append email → waitlist.csv
thank-you.html	Confirmation
serve.py	Local preview server (Python)
.htaccess	Deny public download of CSV

13. Quality & CI

pytest tests/	Unit + API coverage
Orphan golden fixtures	tests/corpus/ground_truth/orphan_spec/
Planted contradiction corpus	Internal engine regression (not customer UI)
Baseline score check	Recall / FP budget on planted set

Known gap: simulate cut/edit lacks a full CI scorecard / labeled golden corpus at scale.

14. Implementation status (July 2026)

--

Structure engine + API	~95% — v1 feature-complete locally
Web workspace	~90% — polish gaps (high-risk badges, docx, mobile)
PDF trustworthiness at Hollywood scale	~65% — needs larger clean benchmark
Deploy / auth / billing	~25% — not revenue-ready
Waitlist landing	Built (Hostinger-ready)
Chrome extension	Not started (v3.1+)

15. Deployment blueprint (target)

Product app

1. Hostinger VPS (or similar): Python 3.10+, venv, spaCy model
2. `uvicorn / run_api.py` behind HTTPS
3. Env: `SESSION_TTL_HOURS`, `CORS_ORIGIN`
4. Single worker (spaCy singleton)
5. Serve `web/` from FastAPI static mount **or** reverse-proxy static separately

Waitlist

1. Upload `landing/*` to `public_html` (or subdomain)
2. PHP enabled; folder writable for `waitlist.csv`
3. Download CSV periodically from File Manager

16. Roadmap (condensed)

Phase A — Pre-beta

- Public URL for API + web
- Wire `.docx` upload
- High-risk scene badges in UI
- Simulate regression runner + small golden set

Phase B — Writer validation

- 10 screenwriters on real Fountain / cleaned PDF
- Measure willingness to pay / clarity of simulate

Phase C — Scale

- Larger simulate + orphan corpora
- Expand Hollywood PDF clean benchmark
- TV pilot/episode edge cases

Later (v3.1+)

- Auth, billing, persistence
- Chrome extension over web editors

- Product decision on contradiction UI

17. Out of scope (do not build in v3.0)

- Plot contradiction customer UI
- Redis / Postgres for sessions
- Stripe / Supabase
- OCR for scanned PDFs
- Native `.fdx` import
- Multi-scene batch simulate
- LLM-written story notes in the structure path

18. How to run (local)

```
cd scriptlensCore
venv\Scripts\Activate.ps1
venv\Scripts\python.exe run_api.py
# Product UI: http://localhost:8000

# Waitlist preview:
venv\Scripts\python.exe landing\serve.py
# http://127.0.0.1:8765/
```

CLI structure-only:

```
venv\Scripts\python.exe run_scriptlens.py path\to\script.fountain --structure-only
```

19. Glossary

OSD	Orphan Scene Detector — weighted story connectivity graph
Orphan	Scene with weak/insufficient outbound narrative links
Simulate cut	Read-only preview of removing one scene
Simulate edit	Read-only preview of rewriting one scene's text
Draft	Mutable working copy; original upload unchanged
Structure mode	<code>full</code> vs <code>limited</code> based on parse/ingest confidence
Continuity graph	Dependency graph used for simulate impact
Planted corpus	Synthetic scripts with known errors for CI

20. Related documents

docs/ARCHITECTURE_v3_STRUCTURE.md	Full v3 engineering specification
docs/SCRIPTLENS_STATUS_REPORT.md	Status, gaps, next actions
docs/ARCHITECTURE.md	Legacy v2.4 (extension + contradictions)
docs/UX_SPEC_v1.md	UX specification
README.md	Setup and quick start

End of ScriptLens Application Blueprint — July 2026